

Goal: Build a hugging face Gradio app to host autogpt, OpenAI codex, perplexity, perplexity pro, pro Gemini, Gemini pro, copilot GitHub DefenseIntelligenceAgency/meta-llama-Llama-3.1-8B-Instruct

Adding Task: Install Gradio library for Python

Adding Task: Import necessary libraries for the Gradio app

Adding Task: Define the input and output interfaces for the app using Gradio

Adding Task: Create a function to define the behavior of the app

Starting task: Install Gradio library for Python

Writing code...

Executing: "Install Gradio library for Python": ```python
!pip install gradio
```

---

Adding Task: Install Gradio library for Python

---

Starting task: Import necessary libraries for the Gradio app

---

Writing code...

---

Adding Task: Import necessary libraries for the Gradio app

---

Executing: "Import necessary libraries for the Gradio app": To build a Gradio app, we need to import the necessary libraries. Gradio is a Python library that allows us to easily create UIs for machine learning models. Here is the code to import the required libraries:

```
```python  
import gradio as gr  
```
```

With this import statement, we can now use the Gradio library to build our app.

---

Adding Task: Create a Gradio interface for the autogpt, OpenAI codex, perplexity, perplexity pro, pro Gemini, Gemini pro, copilot GitHub DefenseIntelligenceAgency/meta-llama-Llama-3.1-8B-Instruct models.

---

Starting task: Create a function to define the behavior of the app

---

Writing code...

---

Executing: "Create a function to define the behavior of the app": ```python

```
import gradio as gr
```

```
import openai
```

```
Set up OpenAI API credentials
```

```
openai.api_key = "YOUR_API_KEY"
```

```
Define the function to generate text using OpenAI Codex
```

```
def generate_text(prompt):
```

```
 response = openai.Completion.create(
```

```
 engine="text-davinci-003",
```

```
 prompt=prompt,
```

```
 max_tokens=100,
```

```
 temperature=0.7,
```

```
 n = 1,
```

```
 stop=None,
```

```
 temperature=0.7
```

```
)
```

```
 return response.choices[0].text.strip()
```

```
Define the behavior of the Gradio app
```

```
def app():
```

```
Define the input and output interfaces of the app
```

```
 inputs = gr.inputs.Textbox(lines=5, label="Prompt")
```

```
 outputs = gr.outputs.Textbox(label="Generated Text")
```

```
Define the function to generate text based on user input
```

```
def generate_text_wrapper(prompt):
```

```
 return generate_text(prompt)
```

```
Create and launch the Gradio app
```

```
 gr.Interface(
```

```
 fn=generate_text_wrapper,
```

```
 inputs=inputs,
```

```
 outputs=outputs,
```

```
 title="OpenAI Codex Text Generation",
```

```
 description="Enter a prompt and view the generated text.",
```

```
).launch()
```

```
Run the app
```

```
app()
```

```
```
```

This code snippet creates a Gradio app that allows users to enter a prompt and generate text using OpenAI Codex. The `generate\_text` function utilizes the OpenAI API to generate text based on the provided prompt. The `app` function sets up the input and output interfaces of the Gradio app, defines a wrapper function to generate text based on user input, and launches the app. The Gradio app includes a textbox for users to enter the prompt and displays the generated text in a separate textbox. The app is titled "OpenAI Codex Text Generation" and has a brief description.

Adding Task: Create a function to define the behavior of the app

Starting task: Define the input and output interfaces for the app using Gradio

Writing code...

Adding Task: Define the input and output interfaces for the app using Gradio

Executing: "Define the input and output interfaces for the app using Gradio": `## Define the input and output interfaces for the app using Gradio`

To define the input and output interfaces for the app using Gradio, we can create a `GradioInterface` class that will handle the user interface and interaction with the models.

First, we need to install the necessary libraries:

```
```python
!pip install gradio
```
```

Next, let's import the required libraries and define the `GradioInterface` class:

```
```python
import gradio as gr

class GradioInterface:
 def __init__(self):
 # Initialize the input and output interfaces
 self.input_interface = None
 self.output_interface = None

 def define_input_interface(self):
 # Define the input interface using Gradio
 self.input_interface = gr.inputs.Textbox(lines=5, label="Input Text")

 def define_output_interface(self):
 # Define the output interface using Gradio
 self.output_interface = gr.outputs.Textbox(label="Output Text")

 def define_interface(self):
 # Define the complete interface combining input and output interfaces
 self.define_input_interface()
 self.define_output_interface()
 return gr.Interface(inputs=self.input_interface, outputs=self.output_interface)
```
```

In the code above, we have defined a `GradioInterface` class that initializes the input and output interfaces as `None`. The `define_input_interface` and `define_output_interface` methods define the input and output interfaces using the Gradio library. In this example, we have used a `Textbox` input interface and a `Textbox` output interface, but you can customize this according to your needs.

The `define_interface` method combines the input and output interfaces and returns a complete Gradio interface that can be used to start the app.

To use the `GradioInterface` class, you can create an instance and call the `define_interface` method to get the complete interface:

```
```python
interface = GradioInterface()
gr_interface = interface.define_interface()
```
```

You can then start the Gradio app by calling the `launch` method on the Gradio interface:

```
```python
gr_interface.launch()
```
```

This will start the app with the defined input and output interfaces, allowing users to input text and view the generated output.

Starting task: Create a Gradio interface for the autogpt, OpenAI codex, perplexity, perplexity pro, pro Gemini, Gemini pro, copilot GitHub DefenseIntelligenceAgency/meta-llama-Llama-3.1-8B-Instruct models.

Writing code...

Executing: "Create a Gradio interface for the autogpt, OpenAI codex, perplexity, perplexity pro, pro Gemini, Gemini pro, copilot GitHub DefenseIntelligenceAgency/meta-llama-Llama-3.1-8B-Instruct models.": To create a Gradio interface for the mentioned models, we need to install the necessary libraries and then write code to integrate Gradio with each model.

First, we need to install the required libraries. Gradio can be installed using pip:

```
```python
pip install gradio
```
```

Once Gradio is installed, we can proceed with integrating the models into Gradio.

Autogpt Model

To integrate the autogpt model into Gradio, we need to define a function that takes an input and returns the model's output. We can then use this function to create a Gradio interface.

```
```python
def autogpt_interface(input_text):
Code to process the input_text using the autogpt model
and generate the output
```
```

```
return output_text
```

```
# Create a Gradio interface for the autogpt model
gradio.Interface(fn=autogpt_interface, inputs="text", outputs="text").launch()
```
```

Replace the comment with the actual code to process the input\_text using the autogpt model and generate the output.

### ## OpenAI Codex Model

The integration process for the OpenAI Codex model is similar to autogpt. We define a function that takes an input and returns the model's output, and then create a Gradio interface using that function.

```
```python
def codex_interface(input_text):
# Code to process the input_text using the OpenAI Codex model
# and generate the output

return output_text

# Create a Gradio interface for the OpenAI Codex model
gradio.Interface(fn=codex_interface, inputs="text", outputs="text").launch()
```
```

Replace the comment with the code to process the input\_text using the OpenAI Codex model and generate the output.

### ## Perplexity Model

For the perplexity model, we need to define a function that calculates the perplexity score for a given input text. We can then create a Gradio interface using this function.

```
```python
def perplexity_interface(input_text):
# Code to calculate perplexity score for the input_text

return perplexity_score

# Create a Gradio interface for the perplexity model
gradio.Interface(fn=perplexity_interface, inputs="text", outputs="text").launch()
```
```

Replace the comment with the code to calculate the perplexity score for the input\_text.

### ## Perplexity Pro Model

The integration process for the perplex

---

Adding Task: Create a Gradio interface for the autogpt, OpenAI codex, perplexity, perplexity pro, pro Gemini, Gemini pro, copilot GitHub DefenseIntelligenceAgency/meta-llama-Llama-3.1-8B-Instruct models.

---

Adding Task: Create a Gradio interface for the DefenseIntelligenceAgency/meta-llama-Llama-3.1-8B-Instruct model.

---

Starting task: Create a Gradio interface for the DefenseIntelligenceAgency/meta-llama-Llama-3.1-8B-Instruct model.

---

Executing: "Summarizing Build a hugging face Gradio app to host autogpt, OpenAI codex, perplexity, perplexity pro, pro Gemini, Gemini pro, copilot GitHub DefenseIntelligenceAgency/meta-llama-Llama-3.1-8B-Instruct": To build a Hugging Face Gradio app to host various models such as autogpt, OpenAI Codex, perplexity, perplexity pro, pro Gemini, Gemini pro, copilot GitHub DefenseIntelligenceAgency/meta-llama-Llama-3.1-8B-Instruct, we need to install the necessary libraries and write code to integrate Gradio with each model.

First, we will install the required libraries. Gradio can be installed using the following command:

```
```python
pip install gradio
```
```

Once Gradio is installed, we can proceed with integrating the models into Gradio.

For the autogpt model, we need to define a function that takes an input and returns the model's output. We can then use this function to create a Gradio interface. Here is an example:

```
```python
def autogpt_interface(input_text):
# Code to process the input_text using the autogpt model
# and generate the output

return output_text

# Create a Gradio interface for the autogpt model
gradio.Interface(fn=autogpt_interface, inputs="text", outputs="text").launch()
```
```

Replace the comment with the actual code to process the input\_text using the autogpt model and generate the output.

Similarly, for the OpenAI Codex model, we define a function that takes an input and returns the model's output. Here is an example:

```
```python
def codex_interface(input_text):
# Code to process the input_text using the OpenAI Codex model
# and generate the output

return output_text
```

```
# Create a Gradio interface for the OpenAI Codex model
gradio.Interface(fn=codex_interface, inputs="text", outputs="text").launch()
```
```

Replace the comment with the code to process the input\_text using the OpenAI Codex model and generate the output.

For the perplexity model, we need to define a function that calculates the perplexity score for a given input text. Here is an example:

```
```python
def perplexity_interface(input_text):
# Code to calculate perplexity score for the input_text

    return perplexity_score

# Create a Gradio interface for the perplexity model
gradio.Interface(fn=perplexity_interface, inputs="text", outputs="text").launch()
```
```

Replace the comment with the code to calculate the perplexity score for the input\_text.

Similarly, for the perplexity pro, pro Gemini, Gemini pro, and copilot GitHub DefenseIntelligenceAgency/meta-llama-a-Llama-3.1-8B-Instruct models, we need to define functions that process the input and generate the respective outputs. We can then create Gradio interfaces for each of these models using the defined functions.

By following this approach, we can build a Hugging Face Gradio app that hosts autogpt, OpenAI Codex, perplexity, perplexity pro, pro Gemini, Gemini pro, copilot GitHub DefenseIntelligenceAgency/meta-llama-Llama-3.1-8B-Instruct, and allows users to interact with these models through a user-friendly interface.

---

Executing: "Response for 'Continue '": ity Pro model is similar to the perplexity model. We define a function that calculates the perplexity score for a given input text and create a Gradio interface using this function.

```
```python
def perplexity_pro_interface(input_text):
# Code to calculate perplexity score for the input_text using the perplexity pro model

return perplexity_score

# Create a Gradio interface for the perplexity pro model
gradio.Interface(fn=perplexity_pro_interface, inputs="text", outputs="text").launch()
```
```

Replace the comment with the code to calculate the perplexity score for the input\_text using the perplexity pro model.

By following this approach, you can integrate each model into Gradio and create separate interfaces for them. Each interface will take an input and generate the respective output based on the model's functionality.

---